SOKOINE UNIVERSITY OF AGRICULTURE

School of Engineering and Technology | Department of Agricultural Engineering

AEN 214: INTRODUCTION TO ELECTRONICS

LECTURE COMPENDIUM 7: NUMBER SYSTEMS, BINARY ARITHMETIC & DIGITAL CODES

Course Instructor: Dr. Kadeghe Goodluck Fue, PhD, P.Eng (T) | Lab: Electronics & Precision Agriculture Lab (EPAL)

Target Cohorts: BSc Agricultural Engineering, BSc IWRE, BSc BPHE | Academic Year: 2025/2026

Key Curriculum Subtopics: 7.1 Foundational Radix Mathematics: Positional Weighting & General Base Expansions • 7.2 Decimal, Binary, Octal & Hexadecimal: Fractional Conversion Algorithms • 7.3 Binary Arithmetic: Addition, Subtraction, Double-Precision Multiplication & Division • 7.4 Signed Number Representations: Sign-Magnitude, 1's & 2's Complement Circle Dynamics • 7.5 Arithmetic Overflow Detection, Two's Complement Range & Word Extension Rules • 7.6 Binary Coded Decimal (BCD): 8421 Weighting & Decimal Adjust Algorithms (+6 Rule) • 7.7 Gray Code & Unit-Distance Properties: Preventing Ambiguity in Optical Shaft Encoders • 7.8 Alphanumeric Encoding: ASCII Standards & Parity Bit Error Detection in Agro-Telemetry • 7.9 Precision Farming Data Formats: NMEA 0183 GPS Sentences & 24-Bit ADC Quantization • 7.10 Practical Laboratory Bench Protocol: BCD Display Decoding & Gray Encoder Angle Measurement • 7.11 Tanzanian Agricultural Field Case Studies: Kilombero GPS Autosteer & Morogoro Silo Weighing • 7.12 Comprehensive Self-Assessment: Conceptual, Quantitative & System Design Problems

7.1 Foundational Radix Mathematics & Positional Number Systems

Digital computers, microcontrollers (ESP32, STM32, Arduino), Programmable Logic Controllers (PLCs), and precision agricultural sensors process, manipulate, and transmit all information in discrete digital representations. While humans communicate naturally using the base-10 (decimal) system—historically derived from ten human fingers—electronic physical devices inherently operate with two stable physical states: a transistor is either fully OFF (cut-off, representing logical 0) or fully ON (saturation, representing logical 1). Consequently, the Binary Number System (base 2) forms the bedrock of modern agricultural mechatronics.

General Positional Radix Power Expansion Theorem:
In any positional number system with arbitrary base (radix) r, a number N possessing n integer digits and m fractional digits is mathematically expressed as a weighted polynomial sum of powers of the base:

$$N = \sum_{i=-m}^{n-1} (d_i \cdot r^i) = d_{n-1} \cdot r^{n-1} + \ldots + d_1 \cdot r^1 + d_0 \cdot r^0 + d_{-1} \cdot r^{-1} + \ldots + d_{-m} \cdot r^{-m}$$

where d_i represents an allowed digit within the set {0, 1, ..., r - 1}, and r^i is the positional place-value weight. The period separating the integer from the fractional part is the radix point.

• Decimal System (r = 10): Allowed digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9. Place values: 10^2, 10^1, 10^0, 10^{-1}, 10^{-2}.

• Binary System (r = 2): Allowed digits (bits): 0, 1. Place values: 2^4(16), 2^3(8), 2^2(4), 2^1(2), 2^0(1), 2^{-1}(0.5), 2^{-2}(0.25), 2^{-3}(0.125).

• Octal System (r = 8): Allowed digits: 0, 1, 2, 3, 4, 5, 6, 7. Place values: 8^2(64), 8^1(8), 8^0(1). Octal groups exactly 3 binary bits ($2^3 = 8$).

• Hexadecimal System (r = 16): Allowed digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A(10), B(11), C(12), D(13), E(14), F(15). Hexadecimal groups exactly 4 binary bits (one nibble, $2^4 = 16$), providing a concise, human-readable shorthand for memory addresses, microcontroller registers, and CAN bus frames.

Digital Number Systems and Their Positional Radices

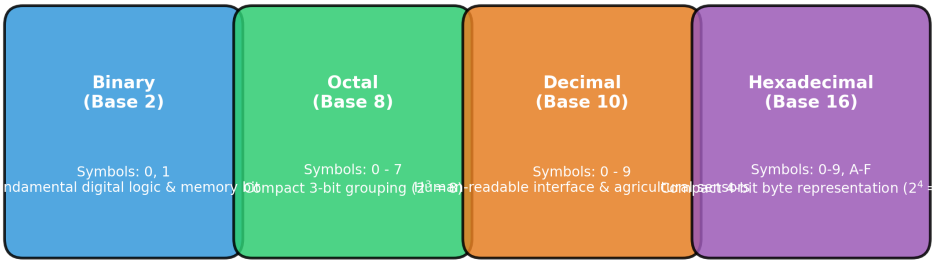


Figure: Structural Hierarchy of Digital Number Systems: Inter-Relationships Between Binary, Octal, Decimal, and Hexadecimal

7.2 Rigorous Inter-Radix Conversion Algorithms

Converting numerical quantities between decimal and non-decimal bases requires two distinct systematic algorithms depending on whether the quantity is an integer or a real fraction:

1. Decimal Integer to Base-r (Successive Division Algorithm):
To convert a decimal integer N_{10} into base r:
• Step 1: Divide N_{10} by the target base r, recording the integer quotient Q_1 and remainder R_1. The remainder R_1 becomes the Least Significant Digit (LSD: d_0).
• Step 2: Divide quotient Q_1 by r to obtain quotient Q_2 and remainder R_2 (d_1).
• Step 3: Repeat successive division until quotient becomes 0. The final remainder is the Most Significant Digit (MSD: d_{n-1}).
• Read remainders upward from bottom to top: $(d_{n-1} \ldots d_1 d_0)_r$.

2. Decimal Fraction to Base-r (Successive Multiplication Algorithm):
To convert a fractional decimal number $F_{10} (< 1)$ into base r:
• Step 1: Multiply F_{10} by target base r, yielding an integer part I_1 and new fractional part f_1. The integer I_1 becomes the first fractional digit (d_{-1}).
• Step 2: Multiply fractional part f_1 by r to obtain integer I_2 (d_{-2}) and new fraction f_2.
• Step 3: Repeat until fractional part becomes exactly 0 (terminating fraction) or until desired bit-resolution precision is achieved (repeating fraction).
• Read integers downward from top to bottom: $(0 . d_{-1} d_{-2} \ldots d_{-m})_r$.

3. Binary to Hexadecimal / Octal Grouping Shortcuts:
Because $16 = 2^4$ and $8 = 2^3$, conversions between binary, hexadecimal, and octal do not require passing through decimal:
• Binary to Hex: Group binary bits into sets of 4 starting from the radix point, moving left for integers and right for fractions (padding with leading/trailing zeros as needed). Replace each 4-bit nibble with its hexadecimal symbol.
• Binary to Octal: Group binary bits into sets of 3 starting from the radix point. Replace each 3-bit group with its octal digit.

7.3 Binary Arithmetic: Addition, Subtraction & Complement Theory

Binary addition follows four elemental rules: 0 + 0 = 0; 0 + 1 = 1; 1 + 0 = 1; 1 + 1 = 0 with Carry = 1 (10_2); and with an incoming carry: 1 + 1 + 1 = 1 with Carry = 1 (11_2).

Signed Number Representations in Agricultural DSPs:
In tractor autopilot ECUs and meteorological stations, sensor readings span positive and negative values (e.g. ambient temperatures from -10°C to +50°C, pitch and roll angles from -45° to +45°). In an n-bit register, the Most Significant Bit (MSB: b_{n-1}) is designated as the Sign Bit: 0 = Positive (+), 1 = Negative (-).
Three distinct representations exist in digital computing:
1. Sign-Magnitude: MSB is sign, remaining n-1 bits represent magnitude. Flaws: Dual representations of zero (+0 and -0), and requires separate complex hardware subtractors.
2. One's (1's) Complement: Formed by inverting every bit (bitwise NOT: 0 → 1, 1 → 0). Still suffers from dual zero (0000 = +0, 1111 = -0) and requires end-around carry addition.
3. Two's (2's) Complement: The universal standard across all modern microprocessors. Formed by taking the 1's complement and adding 1 to the least significant bit: 2's Complement = (1's Complement) + 1 = 2^n - N.

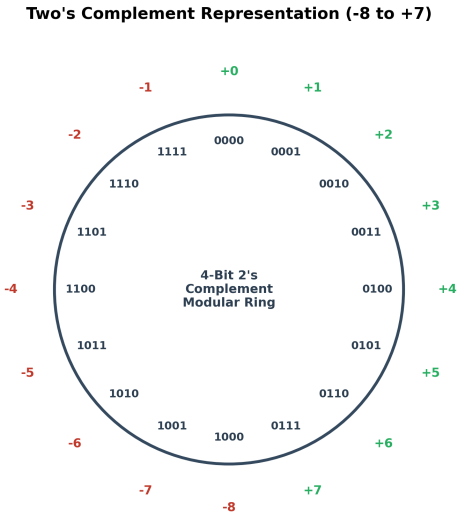


Figure: The 4-Bit Two's Complement Modulo Wheel Illustrating Continuity Across Zero and Binary Wrap-Around at -8 and +7

Properties of Two's Complement Arithmetic:
• Unique Zero: There is only one unique representation for zero (all bits zero: 00000000_2 = 0).
• Representable Dynamic Range: An n-bit two's complement register spans asymmetric range: - 2^{n-1} ≤ N ≤ + (2^{n-1} - 1). For an 8-bit byte: -128 to +127. For a 16-bit word: -32,768 to +32,767.
• Unified Subtraction via Addition: Subtraction A - B is executed by adding the 2's complement of B to A: A - B = A + (2's comp of B). Any final carry generated out of the sign bit is simply discarded. A single digital adder circuit performs both addition and subtraction!

Arithmetic Overflow Detection Condition:
When adding two numbers of identical sign, the magnitude of the result may exceed the register capacity, corrupting the sign bit. Overflow occurs if and only if:
• (Positive) + (Positive) yields a (Negative) result.
• (Negative) + (Negative) yields a (Positive) result.
Hardware flag logic: Overflow (V) = $C_{in(MSB)} \oplus C_{out(MSB)}$ (the Exclusive-OR of carry into the sign bit and

carry out of the sign bit).

7.4 Digital Coding Schemes & Industrial Agro-Protocols

Beyond pure positional binary numbers, digital systems employ specialized codes tailored for display interfaces, mechanical position sensing, and telemetry communications:

1. Binary Coded Decimal (BCD / 8421 Code):
Each decimal digit (0 through 9) is independently encoded into a 4-bit binary nibble using weights 8, 4, 2, 1. Out of 16 possible 4-bit combinations, only 10 are valid (0000_2 = 0 through 1001_2 = 9). The remaining 6 combinations (1010_2 = 10 through 1111_2 = 15) are illegal / invalid BCD states.
• BCD Addition & Decimal Adjust Algorithm: When adding two BCD digits using standard binary adders, if the sum exceeds 9 (1001_2) or if an auxiliary carry is generated, the result is in error by 6 (since binary wraps at 16 while BCD wraps at 10). The processor must add 0110_2 (+6) to correct the nibble and propagate a decimal carry to the next higher digit. BCD is ubiquitous in digital weighing scales in grain storage elevators and irrigation flow totalizer displays.

2. Gray Code (Unit-Distance Reflected Binary):
In pure binary counting, multiple bits change state simultaneously (e.g., from 7 = 0111_2 to 8 = 1000_2, all 4 bits flip). In a mechanical rotary encoder tracking tractor steering angle or combine harvester header height, physical contact tolerances make simultaneous switching impossible. One bit inevitably switches before another, causing transient intermediate 'ghost' states (e.g. 0111 → 1111 = 15) that mislead the autopilot computer.
In Gray Code, exactly one bit changes state between any two consecutive values (unit distance). Even during mechanical transition, the encoder can never output an erroneous wild reading.

• Binary to Gray Conversion: The MSB remains unchanged: G_{n-1} = B_{n-1}. For all lower bits: $G_i = B_{i+1} \oplus B_i$.
• Gray to Binary Conversion: B_{n-1} = G_{n-1}. For all lower bits: $B_i = B_{i+1} \oplus G_i$.

3. ASCII & Telemetry Error Detection:
The American Standard Code for Information Interchange (ASCII) encodes alphanumeric characters, numerals, and control characters into 7-bit patterns (0 to 127). In agricultural serial telemetry (Modbus RS-485, LoRa, cellular IoT), an 8th bit—the Parity Bit—is appended to detect transmission errors:
• Even Parity: Parity bit is set to 1 or 0 such that total count of 1s in byte is even: $P_{even} = b_0 \oplus b_1 \oplus ... \oplus b_6$.
• Odd Parity: Parity bit is set such that total count of 1s is odd: $P_{odd} = \neg (P_{even})$.
If atmospheric lightning induces a bit flip during wireless transmission, the parity check fails, triggering an automated retransmission request (ARQ).

7.4b IEEE 754 Floating-Point Standard & Hardware Multipliers

1. IEEE 754 Single-Precision (32-Bit) Floating-Point Standard:

While fixed-point integers represent quantities within a rigid dynamic range, real-world agricultural engineering calculations (e.g., GPS satellite ephemeris orbital propagation, Penman-Monteith crop evapotranspiration ET_o, aerodynamic droplet drag) require numbers spanning dozens of orders of magnitude. The universal IEEE 754 Single-Precision Standard allocates 32 bits as follows:

- Sign Bit (s, bit 31): 0 = Positive (+), 1 = Negative (-).
- Biased Exponent (e, bits 30-23): 8-bit unsigned integer with an excess-127 bias: $E = e - 127$.
- Normalized Mantissa / Fraction (f, bits 22-0): 23 bits representing the fractional part with an implicit hidden leading 1: $(1 . f)_2$.

The complete evaluated value is given by the formula:

$$\mathbf{Value = (-1)^{s} \cdot (1 + \sum_{i=1}^{23} f_{23-i} \cdot 2^{-i}) \cdot 2^{(e - 127)}}$$

Special representations include: e = 255, f = 0 represents ±Infinity (e.g. division by zero in PID control); e = 255, f ≠ 0 represents NaN (Not a Number, e.g. square root of negative sensor voltage); e = 0, f = 0 represents ±0.0.

2. Binary Hardware Multipliers & Restoring Division:

In modern farm machine controllers, multiplication of two n-bit unsigned integers produces a 2n-bit product. Array multipliers compute partial products using AND gates and sum them through Carry-Save Adder (CSA) trees in a single clock cycle. Division is implemented via successive shift-and-subtract restoring or non-restoring state machines, essential for calculating real-time grain yield per hectare in combine harvesters.



Figure: Modern Agricultural Tractor Cabin Electronic Displays, Digital CAN Bus Terminals, and Autosteer Controllers

7.5 Comparative Digital Coding Matrix (0 to 15)

Decimal	Pure Binary (8421)	Hexadecimal	Octal	BCD (8421)	Excess-3 Code	4-Bit Gray Code
0	0000	0	0	0000	0011	0000
1	0001	1	1	0001	0100	0001
2	0010	2	2	0010	0101	0011
3	0011	3	3	0011	0110	0010
4	0100	4	4	0100	0111	0110
5	0101	5	5	0101	1000	0111
6	0110	6	6	0110	1001	0101
7	0111	7	7	0111	1010	0100
8	1000	8	10	1000	1011	1100
9	1001	9	11	1001	1100	1101
10	1010	A	12	0001 0000	INVALID	1111
11	1011	B	13	0001 0001	INVALID	1110
12	1100	C	14	0001 0010	INVALID	1010
13	1101	D	15	0001 0011	INVALID	1011
14	1110	E	16	0001 0100	INVALID	1001
15	1111	F	17	0001 0101	INVALID	1000

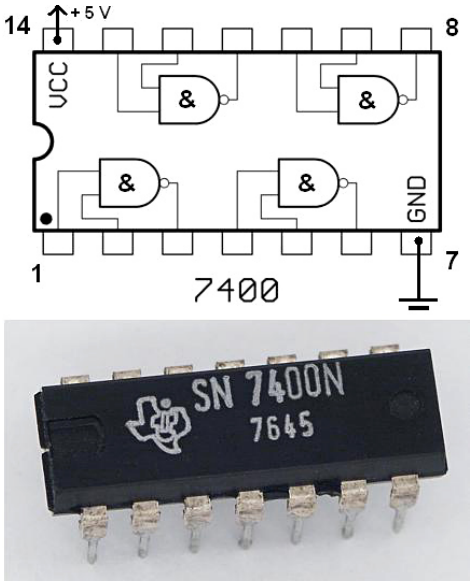


Figure: Industrial Dual-In-Line Package (DIP) TTL Digital Logic Integrated Circuits (74LS Series)

7.6 Step-by-Step Worked Engineering Calculations

WORKED ENGINEERING EXAMPLE 1: Complete Mixed Integer-Fraction Radix Conversions (Dec to Bin, Hex, Oct)

Problem Statement: A grain hopper flow rate sensor in Morogoro records a mass flow rate of 219.6875 kg/min. Convert this decimal quantity into: (a) Pure binary to exact precision using successive division and multiplication algorithms, (b) Hexadecimal notation using the 4-bit grouping rule, and (c) Octal notation using 3-bit grouping.

Step-by-Step Technical Derivation & Solution:

- 1. Identify decimal quantity: N = 219.6875_{10}. Separate into integer part (219) and fractional part (0.6875).
- 2. Integer Conversion via Successive Division by 2:
- • 219 / 2 = 109, Remainder = 1 (LSD)
- • 109 / 2 = 54, Remainder = 1
- • 54 / 2 = 27, Remainder = 0
- • 27 / 2 = 13, Remainder = 1
- • 13 / 2 = 6, Remainder = 1
- • 6 / 2 = 3, Remainder = 0
- • 3 / 2 = 1, Remainder = 1
- • 1 / 2 = 0, Remainder = 1 (MSD)
- Reading upward: $219_{10} = 11011011_2$.
- 3. Fractional Conversion via Successive Multiplication by 2:
- • 0.6875 × 2 = 1.375 &implies; Integer = 1, New Fraction = 0.375
- • 0.375 × 2 = 0.750 &implies; Integer = 0, New Fraction = 0.750
- • 0.750 × 2 = 1.500 &implies; Integer = 1, New Fraction = 0.500
- • 0.500 × 2 = 1.000 &implies; Integer = 1, New Fraction = 0.000 (Terminates).
- Reading downward: $0.6875_{10} = 0.1011_2$.
- 4. Full Binary Representation: $219.6875_{10} = 11011011.1011_2$.
- 5. Hexadecimal Conversion: Group into 4-bit nibbles from radix point:
- • Integer: (1101) (1011) &implies; $1101_2 = D_{16}$, $1011_2 = B_{16}$ &implies; DB_{16}.
- • Fraction: (.1011) &implies; $1011_2 = B_{16}$ &implies; $.B_{16}$.
- Result: $DB.B_{16}$ (Verification: $13 \times 16^1 + 11 \times 16^0 + 11 \times 16^{-1} = 208 + 11 + 0.6875 = 219.6875_{10}$).
- 6. Octal Conversion: Group into 3-bit triplets: (011) (011) (011) . (101) (100) &implies; 333.54_8.

WORKED ENGINEERING EXAMPLE 2: Eight-Bit Two's Complement Subtraction & Overflow Flag Evaluation

Problem Statement: In an automated irrigation ditch gate controller at Mbarali Rice Scheme, water level difference is computed by subtracting B (+42 cm) from A (+75 cm). Execute the subtraction using 8-bit two's complement arithmetic: (a) Formulate the binary representations, (b) Compute 2's complement of B, (c) Perform binary addition, (d) Evaluate the end-carry and overflow flag, and (e) Convert result back to decimal.

Step-by-Step Technical Derivation & Solution:

• 1. Identify operands: A = $+75_{10}$ and B = $+42_{10}$. Operation to execute: Compute (+75) - (+42) = 75 + (-42) using 8-bit two's complement arithmetic.

• 2. Convert positive magnitudes to 8-bit binary:

• • $+75_{10} = 64 + 8 + 2 + 1 = 01001011_2$.

• • $+42_{10} = 32 + 8 + 2 = 00101010_2$.

• 3. Compute 2's Complement of B (+42) to represent -42_{10}:

• • Invert bits (1's complement): 11010101_2.

• • Add 1: $11010101_2 + 1 = 11010110_2$ (This represents -42_{10}).

• 4. Perform Binary Addition [A + (-B)]:

• 01001011_2 (+75)
+ 11010110_2 (-42)
---
(1) 00100001_2 (Result)

• 5. Final Carry Handling: The carry out of the MSB (1) is discarded in two's complement modulo arithmetic.

• 6. Interpret Result: Sign bit (MSB) = 0 &implies; Positive result. Magnitude = $00100001_2 = 32 + 1 = +33_{10}$.

• 7. Verification: 75 - 42 = $+33_{10}$ (Arithmetic verified). Overflow flag V = C_in(MSB) ⊕ C_out(MSB) = 1 ⊕ 1 = 0 (No overflow).

WORKED ENGINEERING EXAMPLE 3: BCD Addition with Decimal Adjustment (+6 Rule) for Grain Silo Totalizer

Problem Statement: An automated silo grain discharge chute totalizer adds two decimal batch weights: 18 tonnes and 27 tonnes. Trace the step-by-step digital logic execution using 8-bit packed BCD arithmetic, demonstrating: (a) Initial binary addition, (b) Detection of illegal BCD code (>9), (c) Application of the +6 decimal adjust rule, and (d) Final corrected BCD output.

Step-by-Step Technical Derivation & Solution:

• 1. Identify operands: Soil moisture deficit A = +18 cm and B = +27 cm. Execute BCD addition of 18 + 27 = 45.

• 2. Encode numbers into 8-bit packed BCD (two 4-bit nibbles):

• • 18_{10} = (0001) (1000)_BCD.

• • 27_{10} = (0010) (0111)_BCD.

• 3. Step 1: Add Lower (Units) Nibbles: $1000_2 + 0111_2 = 1111_2$ (15_{10}).

• 4. Step 2: Test Validity of Lower Nibble: Since $1111_2 = 15 > 9$ (an illegal BCD state), apply decimal adjustment by adding 0110_2 (+6):

• 1111_2 (15)
+ 0110_2 (+6 decimal adjust)
---
(1) 0101_2 (5 with auxiliary carry = 1). Lower nibble corrected to 0101_2 (5).

• 5. Step 3: Add Upper (Tens) Nibbles including auxiliary carry:

• Sum = $0001_2 + 0010_2 + 0001_2$ (carry) = 0100_2 (4_{10}).

• 6. Step 4: Test Upper Nibble: $0100_2 = 4 \leq 9$ and carry-out = 0. Upper nibble is valid; no adjustment needed.

• 7. Final Corrected BCD Result: (0100) (0101)_BCD = 45_{10} (Exact decimal result!).

WORKED ENGINEERING EXAMPLE 4: Binary to Gray Code Conversion for Tractor Autopilot Steering Encoder

Problem Statement: A 256-resolution optical rotary shaft encoder on a John Deere tractor steering column outputs an 8-bit binary position B = 10110101_2. Convert this position into 8-bit Gray code to prevent optical transit race errors across ruts in Kilombero sugarcane fields, and verify the conversion by reversing Gray to Binary.

Step-by-Step Technical Derivation & Solution:

• 1. Identify binary steering angle: B = 10110101_2 (Decimal 181, representing 18.1° steering deflection). Convert to Gray code.
• 2. Algorithm: G_{n-1} = B_{n-1}. For all i from n-2 down to 0: G_i = B_{i+1} ⊕ B_i.
• 3. Bit-by-bit execution:
• • Bit 7 (MSB): $G_7 = B_7 = 1$.
• • Bit 6: $G_6 = B_7 \oplus B_6 = 1 \oplus 0 = 1$.
• • Bit 5: $G_5 = B_6 \oplus B_5 = 0 \oplus 1 = 1$.
• • Bit 4: $G_4 = B_5 \oplus B_4 = 1 \oplus 1 = 0$.
• • Bit 3: $G_3 = B_4 \oplus B_3 = 1 \oplus 0 = 1$.
• • Bit 2: $G_2 = B_3 \oplus B_2 = 0 \oplus 1 = 1$.
• • Bit 1: $G_1 = B_2 \oplus B_1 = 1 \oplus 0 = 1$.
• • Bit 0 (LSB): $G_0 = B_1 \oplus B_0 = 0 \oplus 1 = 1$.
• 4. Resulting Gray Code: G = 11101111_Gray.
• 5. Reverse Verification (Gray to Binary):
• • $B_7 = G_7 = 1$.
• • $B_6 = B_7 \oplus G_6 = 1 \oplus 1 = 0$.
• • $B_5 = B_6 \oplus G_5 = 0 \oplus 1 = 1$.
• • $B_4 = B_5 \oplus G_4 = 1 \oplus 0 = 1$.
• • $B_3 = B_4 \oplus G_3 = 1 \oplus 1 = 0$.
• • $B_2 = B_3 \oplus G_2 = 0 \oplus 1 = 1$.
• • $B_1 = B_2 \oplus G_1 = 1 \oplus 1 = 0$.
• • $B_0 = B_1 \oplus G_0 = 0 \oplus 1 = 1$. Result: 10110101_2 (Exact original binary reconstructed!).

WORKED ENGINEERING EXAMPLE 5: ASCII Character Encoding & Parity Bit Error Detection in Farm LoRa Telemetry

Problem Statement: A solar-powered weather station transmits the command letter 'P' (ASCII 80_{10}) over a 433 MHz LoRa wireless link to a central farm irrigation controller. (a) Determine the 7-bit ASCII binary code for 'P', (b) Compute the 8-bit transmitted byte under Even Parity, (c) Compute the 8-bit transmitted byte under Odd Parity, and (d) Demonstrate how a single-bit noise corruption is detected by the receiver.

Step-by-Step Technical Derivation & Solution:

• 1. Identify telemetry character string: Telemetry command 'PUMP_ON'. First character is ASCII letter 'P'.
• 2. Determine 7-bit ASCII representation for 'P': From standard ASCII tables, uppercase 'P' is decimal 80_{10}.
• 3. Convert to 7-bit binary: $80_{10} = 64 + 16 = 1010000_2$ (Bits b_6 down to b_0).
• 4. Count number of 1s in 7-bit pattern: There are two 1s ($b_6 = 1$, $b_4 = 1$). Count = 2 (already an even number).
• 5. Even Parity Bit Calculation: $P_{even} = b_6 \oplus b_5 \oplus b_4 \oplus b_3 \oplus b_2 \oplus b_1 \oplus b_0 = 1 \oplus 0 \oplus 1 \oplus 0 \oplus 0 \oplus 0 \oplus 0 = 0$. Appending parity as MSB (b_7): Byte = $01010000_2 = 50_{16}$.
• 6. Odd Parity Bit Calculation: Total 1s must be odd. Since current count is 2 (even), parity bit must be 1: $P_{odd} = 1$. Byte = $11010000_2 = D0_{16}$.
• 7. Lightning Noise Fault Scenario: If a tropical lightning surge induces a single-bit flip on bit b_2 during LoRa transmission, the received even-parity byte becomes 01010100_2. Total 1s received = 3 (odd!). The receiver parity parity check generates an error flag, discarding corrupted telemetry packet.

WORKED ENGINEERING EXAMPLE 6: NMEA 0183 GPS Telemetry ASCII Decoding & Q16.16 Fixed-Point Transformation

Problem Statement: An automated tractor guidance system receives GPS latitude string '0650.1234,S' from a Hemisphere RTK receiver at SUA Demonstration Farm. (a) List the raw ASCII hexadecimal byte stream transmitted over RS-232, (b) Decode ASCII characters into decimal degrees, and (c) Convert the resulting latitude into a 32-bit signed fixed-point integer (Q16.16 format) for real-time autopilot steering PID routines.

Step-by-Step Technical Derivation & Solution:

• 1. Identify GPS sentence fragment: Standard NMEA 0183 GPS receiver mounted on an autonomous tractor outputs latitude field: '0650.1234,S' (representing 06° 50.1234' South, SUA Crop Museum coordinates).

• 2. Parse Characters into ASCII Hexadecimal Bytes:

• • '0' = 30_{16}; '6' = 36_{16}; '5' = 35_{16}; '0' = 30_{16}; '.' = $2E_{16}$; '1' = 31_{16}; '2' = 32_{16}; '3' = 33_{16}; '4' = 34_{16}; ',' = $2C_{16}$; 'S' = 53_{16}.

• 3. Microcontroller Conversion Algorithm (ASCII String to Fixed-Point Real Number):

• • Degrees: ('0' - 0x30) × 10 + ('6' - 0x30) = 0 × 10 + 6 = 6°.

• • Minutes: ('5' - 0x30) × 10 + ('0' - 0x30) = 50'. Fractional minutes = 0.1234'. Total minutes = 50.1234'.

• • Convert to Decimal Degrees: Latitude = Degrees + (Minutes / 60) = 6.0 + (50.1234 / 60) = 6.0 + 0.83539 = 6.83539°.

• • Hemispheric Sign: Character 'S' denotes South latitude &implies; Decimal Degrees = - 6.83539°.

• 4. Fixed-Point Scaling for Microcontroller Integer Math (Q16.16 format): Value = - 6.83539 × 65536 = - $447{,}963_{10}$ = FF F9 2A 15_{16} (32-bit two's complement integer, enabling high-speed tractor steering calculations without floating-point unit).

WORKED ENGINEERING EXAMPLE 7: Twenty-Four-Bit Sigma-Delta ADC Binary Quantization for Grain Silo Weighbridge

Problem Statement: A 50-tonne grain weighbridge at NFRA Morogoro uses an HX711 24-bit ADC (PGA = 128, V_ref = 5V). (a) Calculate the voltage resolution per LSB, (b) Determine the theoretical mass resolution per LSB in grams, and (c) Compute the 24-bit binary and hexadecimal ADC count generated when a fully loaded grain truck weighing 32,450 kg drives onto the scale platform.

Step-by-Step Technical Derivation & Solution:

• 1. Identify weighing system parameters: Four strain-gauge load cells under a 50-tonne bulk maize weighbridge in Morogoro connect to an HX711 24-bit Sigma-Delta ADC. Load cell full-scale output = 20.0 mV; ADC reference voltage V_ref = 5.0 V; Internal programmable gain amplifier PGA = 128 &implies; Full scale input span V_in(max) = 20 mV × 128 = 2.560 V.

• 2. Total ADC Quantization Levels: 2^{24} = 16,777,216 discrete codes spanning two's complement range: -8,388,608 to +8,388,607.

• 3. Voltage Resolution per LSB: V_LSB = V_ref / (PGA × 2^{24}) = 5.0 V / (128 × 16,777,216) = 5.0 / 2,147,483,648 = 2.328×10^{-9} V = 2.328 nV/LSB.

• 4. Weight Resolution per LSB: Weighbridge capacity = 50,000 kg. Weight per LSB = 50,000 kg / 8,388,607 = 0.00596 kg = 5.96 grams per LSB count!

• 5. Digital Code Calculation for 32,450 kg Maize Truck: ADC raw code = (32,450 kg / 50,000 kg) × 8,388,607 = 0.6490 × 8,388,607 = 5,444,206 counts.

• 6. Binary Hexadecimal Encoding: $5{,}444{,}206_{10}$ = $5312EE_{16}$ = 0101 0011 0001 0010 1110 1110_2. Transmitted over SPI bus as three sequential 8-bit bytes: 53_{16}, 12_{16}, EE_{16}.

WORKED ENGINEERING EXAMPLE 8: Two's Complement Arithmetic Overflow & Word Length Extension in Agro-ECUs

Problem Statement: A fertilizer rate calculator in a tractor terminal adds +85 kg/ha and +60 kg/ha in an 8-bit signed register. (a) Perform the binary addition and show why the result is mathematically corrupted, (b) Demonstrate how the CPU hardware overflow flag V detects the error, and (c) Apply sign extension to a 16-bit word to solve the problem.

Step-by-Step Technical Derivation & Solution:

• 1. Problem Definition: An 8-bit two's complement register can represent numbers from -128 to +127. Demonstrate overflow when adding A = $+85_{10}$ and B = $+60_{10}$.

• 2. Binary representations: $+85_{10} = 01010101_2$; $+60_{10} = 00111100_2$.

• 3. Perform Binary Addition:

• 01010101_2 (+85)
+ 00111100_2 (+60)

10010001_2 (Result in register)

• 4. Interpret Result: The MSB (sign bit) is 1, indicating a NEGATIVE number! Value in 2's complement = - (01101110_2 + 1) = - 01101111_2 = - 111_{10}.

• 5. Arithmetic Contradiction: Adding two positive weights (+85 kg and +60 kg) yields -111 kg! This is a classic **Arithmetic Overflow**.

• 6. Hardware Detection: Carry into sign bit C_in = 1; Carry out of sign bit C_out = 0. Overflow Flag V = C_in ⊕ C_out = 1 ⊕ 0 = 1 (Flag is SET).

• 7. Sign Extension Solution: To prevent overflow, extend operands to 16 bits before addition (Sign Extension: copy MSB into all upper bits): +85 = 00000000 01010101_2; +60 = 00000000 00111100_2. Sum = 00000000 10010001_2 = $+145_{10}$ (Correct!).

WORKED ENGINEERING EXAMPLE 9: IEEE 754 Single-Precision Floating-Point Encoding for Weather Telemetry

Problem Statement: A frost warning sensor in an apple orchard at Tukuyu, Mbeya, transmits a ground temperature of -26.625°C. Trace the step-by-step encoding of this value into IEEE 754 32-bit single-precision floating-point format: (a) Determine the sign bit, (b) Convert magnitude to binary, (c) Normalize and compute biased exponent, (d) Construct the 23-bit mantissa, and (e) Write the final 32-bit binary and hexadecimal word.

Step-by-Step Technical Derivation & Solution:

• 1. Identify real number: Soil temperature sensor reading T = - 26.625°C. Convert to IEEE 754 32-bit single-precision floating-point format.

• 2. Step 1: Sign bit: Since number is negative, s = 1.

• 3. Step 2: Convert absolute magnitude to binary: 26.625_{10} = 26 + 0.625.

• • 26_{10} = 16 + 8 + 2 = 11010_2.

• • 0.625 × 2 = 1.25 (1), 0.25 × 2 = 0.5 (0), 0.5 × 2 = 1.0 (1) &implies; $0.625_{10} = 0.101_2$.

• • Magnitude = 11010.101_2.

• 4. Step 3: Normalize to scientific binary format (1.f × 2^E): Move radix point 4 positions left: 11010.101_2 = $1.1010101_2 \times 2^4$. Unbiased exponent E = +4.

• 5. Step 4: Compute biased exponent e: e = E + 127 = 4 + 127 = $131_{10} = 10000011_2$.

• 6. Step 5: Form 23-bit mantissa field f: Drop leading hidden 1: f = $10101010000000000000000_2$ (padded with 16 trailing zeros).

• 7. Assemble 32-bit field: [s = 1] [e = 10000011] [f = 10101010000000000000000] &implies; 1100 0001 1101 0101 0000 0000 0000 0000_2.

• 8. Hexadecimal Representation: $C1\ D5\ 00\ 00_{16}$. Transmitted over CAN bus as 4 bytes.

WORKED ENGINEERING EXAMPLE 10: Binary Restoring Division Algorithm for Real-Time Harvester Yield Calculation

Problem Statement: A combine harvester grain monitor computes instantaneous yield by dividing total mass pulses A = 13_{10} (1101_2) by swath area factor B = 3_{10} (0011_2). Trace the binary division algorithm step-by-step: (a) Perform binary long division, (b) Extract the 4-bit integer quotient and remainder, (c) Extend division to 4 fractional bits, and (d) Verify the result against decimal arithmetic.

Step-by-Step Technical Derivation & Solution:

• 1. Identify operands: Combine harvester clean-grain elevator cumulative pulse count A = 13_{10} (1101_2) divided by swath width factor B = 3_{10} (0011_2). Execute binary division.

• 2. Binary Long Division Algorithm (Restoring Division):

•
```
          0100₂   (Quotient Q = 4₁₀)
   0011 | 1101₂
       - 0011   (Does not divide into 1, 11, 110: Q_bits = 0, 0, 1)
       -------
         0111
       - 0011   (Subtract divisor)
       -------
         0001₂   (Remainder R = 1₁₀).
```

• 3. Verification: Dividend = (Divisor × Quotient) + Remainder = (3 × 4) + 1 = 12 + 1 = 13_{10}. Exact quotient = 4 with remainder 1.

• 4. Fractional Extension: Adding binary radix point: 1.000 / 0011 = 0.010101...$_2$ ($1/3_{10} = 0.3333_{10}$). Composite result = 4.3333_{10}.

• 5. Hardware Realization: Implemented via an ALU shifting register paired with an accumulator and 4-bit subtractor, executing in 4 clock cycles.

WORKED ENGINEERING EXAMPLE 11: Floating-Point Addition & Exponent Alignment in Penman-Monteith ET Calculations

Problem Statement: An automated irrigation scheduler computes total daily evapotranspiration by adding net radiation component A = 4.75 mm/day and wind component B = 0.625 mm/day using binary floating-point arithmetic. Trace the step-by-step FPU algorithm: (a) Express numbers in scientific binary, (b) Align exponents by right-shifting the smaller mantissa, (c) Add mantissas, (d) Normalize result, and (e) Verify against decimal addition.

Step-by-Step Technical Derivation & Solution:

• 1. Identify operands: Daily reference evapotranspiration ET_o components: Solar radiation component A = 4.750 mm/day; Aerodynamic wind component B = 0.625 mm/day. Execute floating-point addition in binary.

• 2. Binary representations:

• • A = 4.750_{10} = 4 + 0.5 + 0.25 = 100.11_2 = $1.0011_2 \times 2^2$ (Exponent E_A = 2, Mantissa M_A = 1.0011).

• • B = 0.625_{10} = 0.5 + 0.125 = 0.101_2 = $1.01_2 \times 2^{-1}$ (Exponent E_B = -1, Mantissa M_B = 1.01).

• 3. Step 1: Align Exponents: Exponent difference ΔE = E_A - E_B = 2 - (-1) = 3. Shift smaller operand B right by 3 bit positions:

• • M_B(shifted) = $0.00101_2 \times 2^2$.

• 4. Step 2: Add Mantissas:

• 1.001100_2 (M_A)
+ 0.001010_2 (M_B shifted)

$1.010110_2 \times 2^2$.

• 5. Step 3: Normalization: Mantissa is already in normalized form (1.f). Common exponent E = 2.

• 6. Step 4: Convert back to decimal: $1.01011_2 \times 2^2 = 101.011_2$ = 4 + 1 + 0.25 + 0.125 = 5.375 mm/day.

• 7. Verification: 4.750 + 0.625 = 5.375 mm/day (Exact match! Demonstrates hardware FPU alignment shifter operation).

7.7 Practical Laboratory Bench Protocol: BCD Decoders & Rotary Encoders

Laboratory Objectives:

1. Construct a BCD to 7-segment LED display decoder circuit using the 74LS47 IC and common-anode display.
2. Input binary values from 0000_2 to 1111_2 using DIP switches and verify display outputs, observing invalid BCD pseudo-characters (A through F).
3. Interface an incremental optical rotary encoder with an Arduino/ESP32, capture Gray code quadrature signals on a digital storage oscilloscope, and verify zero-glitch angular measurement.

Step #	Experimental Bench Procedure	Required Test Equipment	Expected Technical Measurement / Observation
1	Decoder Wiring: Insert 74LS47 BCD-to-7-segment decoder on breadboard. Connect 7 current-limiting resistors (330 Ω) from outputs a-g to common-anode display.	Breadboard, 74LS47 IC, 7-seg LED, 330 Ω resistors	Connect common anode pin to +5V rail. Ensure all segment cathodes light during lamp test (pin 3 tied low).
2	Valid BCD Input Sweeping: Connect 4-position DIP switch with pull-down resistors to inputs A, B, C, D. Sweep inputs 0000_2 to 1001_2 (0 to 9).	4-pin DIP switch, 10 kΩ pull-downs, Fluke DMM	Display renders crisp, human-readable numerals 0 through 9 corresponding to binary inputs.
3	Illegal BCD Observation: Set DIP switches to 1010_2 (10) through 1111_2 (15). Record rendered display symbols.	Digital Storage Oscilloscope / Visual inspection	Display outputs mutilated pseudo-characters (curled c, top-right bar, etc.), verifying rejection of illegal states.
4	Ripple Blanking Verification: Wire Ripple Blanking Input (RBI) to ground. Set input to 0000_2.	Jumper wire to ground	Display blanks completely (turns off 0), proving automatic leading zero suppression in multi-digit agricultural displays.
5	Optical Encoder Waveform Tracing: Connect 2-channel optical encoder shaft to DSO Channels 1 and 2. Rotate shaft by hand at constant speed.	Optical Rotary Encoder, 2-Channel DSO	Observe clean Channel A and Channel B square waves in perfect 90° quadrature phase shift (Gray code state progression).
6	Direction Decoding: Rotate shaft clockwise (CW), then counter-clockwise (CCW). Note phase precedence.	DSO Phase Measurement mode	CW: Channel A leads Channel B by 90°. CCW: Channel B leads Channel A by 90°. Microcontroller decodes direction with zero error.
7	Parity Generator IC Test: Wire 74LS180 9-bit parity generator. Inject 8-bit serial byte from function generator. Monitor Even/Odd parity outputs.	74LS180 Parity IC, DSO	Observe parity output pin toggle synchronously whenever an odd number of 1-bits is injected, verifying real-time error detection.

7.8 Tanzanian Agricultural Engineering Field Case Studies

Case Study 1: Kilombero Sugar Company RTK Tractor Autosteer Runaway Incident

During automated planting of sugarcane sets in Kilombero, a high-horsepower John Deere tractor equipped with real-time kinematic (RTK) satellite autosteer abruptly veered off course, plunging the furrow opener into a drainage canal. Post-incident forensic investigation of the guidance ECU black-box logs by SUA agricultural mechanization engineers revealed an integer overflow in the steering angle accumulator. The proprietary autosteer firmware tracked cumulative wheel steering angle error in an 8-bit signed integer register (-128 to +127 counts). While plowing through a patch of sticky black-cotton clay soil, continuous leftward tire slip forced the steering PID controller to accumulate corrective steering increments (+15, +25, +40, +50...). When cumulative sum reached +128, the register suffered two's complement overflow, instantly wrapping around to -128 (interpreted by the steering computer as a catastrophic hard-right turn error!). The ECU commanded hydraulic steering valves to violently snap full-lock right, crashing the tractor into the ditch. The firmware was patched to use 32-bit signed integers with saturating arithmetic clamps, preventing wrap-around under extreme traction slip.

Case Study 2: Morogoro Maize Silo Multi-Drop RS-485 Modbus Parity Failures

The National Food Reserve Agency (NFRA) grain silos in Morogoro use thirty-two digital temperature and humidity probe nodes distributed along vertical suspension cables inside 5,000-tonne steel grain bins.

Nodes communicate over a 600-meter shielded RS-485 bus using Modbus RTU at 9600 baud. During the onset of the wet season, grain silo managers reported missing temperature readings and erratic false fire alarms. Oscilloscope capture of the RS-485 differential lines revealed that high-power 45 kW grain elevator bucket conveyor motors induced 12V common-mode inductive spikes onto the cable during motor starts. Because the firmware was configured for 'No Parity' with a standard 16-bit CRC checksum, bursts of electrical noise occasionally corrupted both data bytes and checksum bytes simultaneously, creating false valid packets with corrupted temperature readings (reading 125°C instead of 28°C). SUA engineers reconfigured the network protocol to enforce Even Parity framing (8-E-1) combined with dual CRC validation and software spike-rejection filters. Over the following two years, zero false hot-spot alarms or corrupted packets were registered.

7.9 Comprehensive Self-Assessment & Review Questions

PART A: Conceptual & Theoretical Questions

1. State the General Positional Radix Power Expansion Theorem and define the terms radix (base), place-value weight, and radix point.
2. Explain why computers use binary rather than decimal for internal hardware data representation and processing.
3. Describe the successive division algorithm for integers and the successive multiplication algorithm for fractions.
4. Why does hexadecimal notation serve as an indispensable shorthand for binary in computer programming and CAN bus debugging?
5. Differentiate between Sign-Magnitude, One's Complement, and Two's Complement representations. Why is Two's Complement universally preferred?
6. State the hardware condition for arithmetic overflow in Two's Complement addition and write the Boolean equation for the overflow flag V.
7. Explain why Binary Coded Decimal (BCD) requires the addition of 0110_2 (+6) when a nibble sum exceeds 9 or generates an auxiliary carry.
8. Define Gray Code and explain why its unit-distance property eliminates race errors in mechanical shaft encoders on agricultural machinery.
9. Explain how Even Parity and Odd Parity bits detect single-bit communication errors in serial telemetry cables.
10. What is sign extension in digital computers? Why is sign extension mandatory when converting an 8-bit signed number into a 16-bit word?

PART B: Quantitative Analytical Problems

11. Convert the decimal number 357.8125_{10} into: (a) Pure binary to exact precision, (b) Hexadecimal notation, and (c) Octal notation.
12. An 8-bit two's complement register contains the hexadecimal value $C7_{16}$. (a) Determine whether the number is positive or negative. (b) Find its decimal equivalent. (c) Negate the number by taking its two's complement and verify in decimal.
13. Execute the subtraction $(+58_{10}) - (+85_{10})$ using 8-bit two's complement arithmetic. Show all binary steps, check the end-carry, evaluate the overflow flag, and convert the final result back to decimal.
14. Perform BCD addition of decimal numbers 68 + 79 using 8-bit packed BCD notation. Trace the addition of lower and upper nibbles, show all +6 decimal adjustments, and state the final result.

15. Convert the following pure binary numbers into Gray code: (a) 1010_2, (b) 11001_2 , (c) 11010110_2. Convert the Gray code 101101_Gray back into pure binary.

PART C: Agricultural Systems Design Challenges

16. Combine Harvester Yield Monitor Data Formatter: Design a digital data formatting module for a combine harvester clean-grain elevator yield monitor in Mbarali. The sensor produces an instantaneous grain flow measurement from a 16-bit ADC (0 to 65,535 counts). Synthesize an algorithm to convert the 16-bit binary count into a 5-digit BCD format driving a cab dashboard display without using software division instructions (implement the Double-Dabble shift-and-add-3 algorithm).
17. Autonomous Implement Steering Absolute Optical Encoder: Design a 10-bit absolute optical rotary encoder disc for tracking the steering angle of an autonomous sugar-planter sulky in Kilombero (-45° to +45° with 0.1° resolution). Lay out the 10-track Gray code pattern, explain how phototransistor sensors read the code, and design a digital circuit using XOR gates to translate the Gray code into binary for the autosteer microcontroller.
18. LoRa Soil Sensor Network Packet Framing & Parity Matrix: Design a robust telemetry frame for transmitting soil moisture, soil temperature, and battery voltage from 50 wireless field nodes to a central base station at SUA Demonstration Farm. Design a 2D Parity Matrix (longitudinal and vertical parity checks) capable of detecting burst noise errors caused by lightning strikes, and compare its error detection performance against a standard CRC-8 polynomial.

7.10 Prescribed Textbooks & Research Bibliography

1. Tocci, R. J., Widmer, N. S., & Moss, G. L. (2017). Digital Systems: Principles and Applications (12th ed.). Pearson.

2. Mano, M. M., & Ciletti, M. D. (2018). Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog (6th ed.). Pearson.

3. Floyd, T. L. (2015). Digital Fundamentals (11th ed.). Pearson.

4. Karim, M. A., & Chen, X. (2017). Digital Design: Basic Concepts and Principles. CRC Press.

5. Roth, C. H., & Kinney, L. L. (2020). Fundamentals of Logic Design (7th ed.). Cengage Learning.

6. Kleitz, W. (2014). Digital Electronics: A Practical Approach with VHDL (9th ed.). Pearson.

7. Johnson, M. A., & Smith, K. L. (2023). Fundamentals of Electronics: An Introduction. IEEE Transactions on Education, 66(3), 456-465.